UNIVERSITY OF WESTMINSTER

Department of Computer Science

CW1: INDIVIDUAL SUBMISSION

3D Interactive Media – 5MMCS007W

RABBIT SURVIVAL LEGO

Student name	Udula Liyanaarachchi
Westminster ID	W2181814
Coursework	CW1 – Individual Submission
Group	02
Project	Rabbit Survival LEGO
GitHub branch	student-1/falling-rocks

Individual contribution report

Table of Contents

1. Project overview 3
2. Individual contribution and responsibilities 3
3. Falling-rock system 3
4. Rabbit health and damage 4
5. Bridge and collision behaviour 5
6. Cave victory trigger 6
7. Testing and debugging 7
8. GitHub contribution and collaboration 7
9. Personal reflection 8
10. AI assistance summary 8

1. Project overview

Rabbit Survival LEGO is a Unity-based 3D interactive game prototype. The player controls a rabbit and progresses through an environment containing obstacles and interactive areas. The project uses Unity gameplay systems such as physics, colliders, input, triggers and interface feedback to make the environment interactive.

This individual submission focuses on my contribution to four gameplay areas: the falling-rock system, rabbit health and damage, bridge collision behaviour, and the cave victory trigger. The work was developed and tested as part of Group 02 for 5MMCS007W - 3D Interactive Media, Coursework 1.

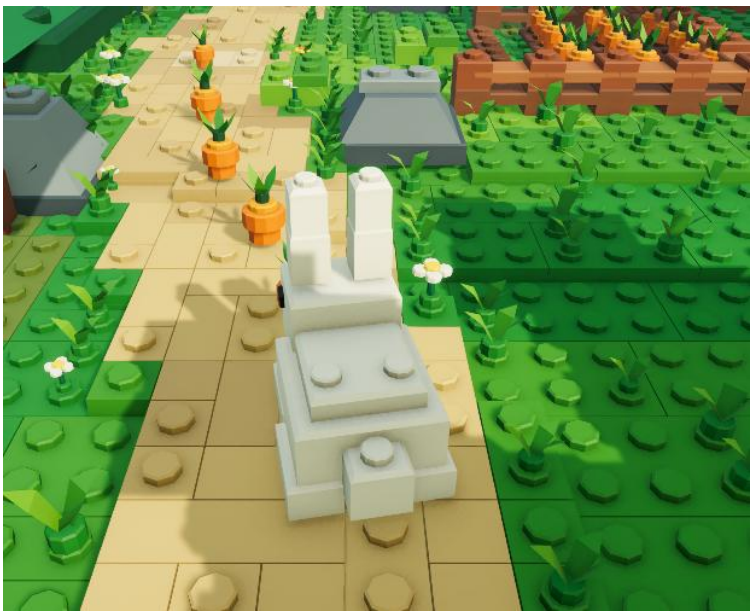


Figure 1 - Rabbit Survival

2. Individual contribution and responsibilities

My contribution covered the falling-rock feature, the rabbit health and damage feature, bridge collision setup, and the cave victory trigger. The related project components include RockSpawner.cs, FallingRock.cs, RabbitHealth.cs, the bridge collider configuration and CaveWinTrigger.cs.

I worked on these features as connected parts of the gameplay. The falling rocks provide an environmental hazard, the health system represents the effect of damage, the bridge colliders define the physical route through the level, and the cave trigger provides the completion condition. I tested the features in Unity. My individual GitHub work is associated with the student-1/falling-rocks branch.

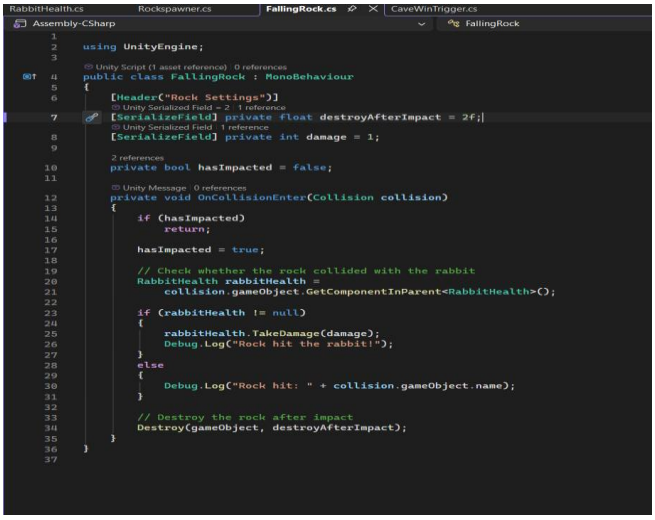
3. Falling-rock system

The falling-rock feature is represented by RockSpawner.cs and FallingRock.cs. The spawner is responsible for creating the falling-rock hazard, while the falling-rock behaviour script handles the spawned object during gameplay. The Unity project evidence includes the falling-rock prefab and its Rigidbody and Box Collider configuration.

During implementation and testing, the important setup points were the prefab reference, the spawning position and timing, and the physical components attached to the falling object. The Rigidbody allows the object to participate in Unity physics, while the collider allows it to detect physical contact. The final values and exact collision response should be read from the submitted script and Inspector configuration.



Figure 2 - Falling Rock



```
RabbitHealth.cs   Rockspawner.cs   FallingRock.cs   CaveWinTrigger.cs
Assembly-CSharp                                    FallingRock

using UnityEngine;

Unity Script (1 asset reference) 0 references
public class FallingRock : MonoBehaviour
{
    [Header("Rock Settings")]
    Unity Serialized Field 2 1 reference
    [SerializeField] private float destroyAfterImpact = 2f;
    Unity Serialized Field 1 reference
    [SerializeField] private int damage = 1;

    2 references
    private bool hasImpacted = false;

    Unity Message 0 references
    private void OnCollisionEnter(Collision collision)
    {
        if (hasImpacted)
            return;

        hasImpacted = true;

        // Check whether the rock collided with the rabbit
        RabbitHealth rabbitHealth =
            collision.gameObject.GetComponentInParent<RabbitHealth>();

        if (rabbitHealth != null)
        {
            rabbitHealth.TakeDamage(damage);
            Debug.Log("Rock hit the rabbit!");
        }
        else
        {
            Debug.Log("Rock hit: " + collision.gameObject.name);
        }

        // Destroy the rock after impact
        Destroy(gameObject, destroyAfterImpact);
    }
}
```

Figure 3 - Falling Rock C#

4. Rabbit health and damage

RabbitHealth.cs is the script associated with the rabbit health and damage feature. The feature was tested in Unity, and the available evidence includes Console output showing health reduction and the defeat state. This provides evidence that damage is applied during gameplay and that the health system responds when the rabbit reaches the defeat condition.

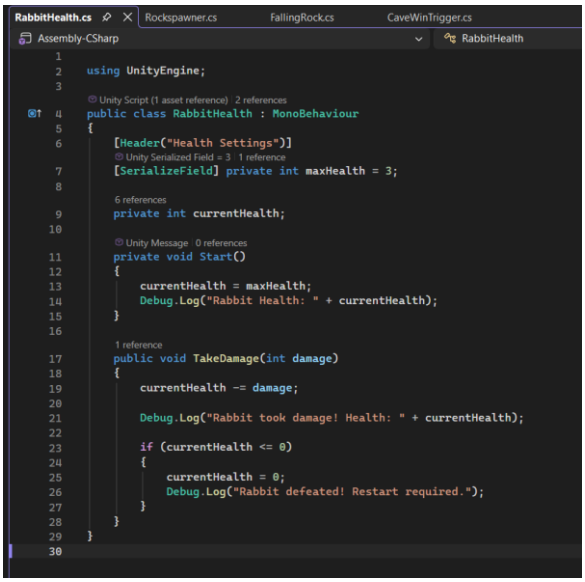


Figure 4 – RabbitHealth.cs

The health feature works alongside the environmental hazard system: the player can encounter hazards, receive damage and reach a defeat state. The exact starting health, damage amount, and any restart behaviour should be stated using the final submitted script and observed build values rather than assumed values.

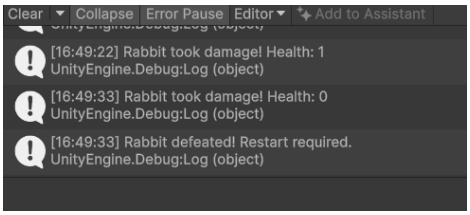


Figure 5 - Rabbit Health Console Output

5. Bridge and collision behaviour

The bridge section uses collider components to define the walkable route and the boundaries of the environment. My contribution included the bridge collision setup and testing the rabbit moving across the bridge. The evidence collected for the project includes the bridge hierarchy and collider configuration, as well as the rabbit navigating across the bridge.

Colliders are important because they define where the player and other physics objects can make contact with the environment. The bridge setup therefore needs to be considered together with the rabbit controller and any Rigidbody-based objects in the scene. The final report should describe the actual collider arrangement visible in the submitted Unity scene.

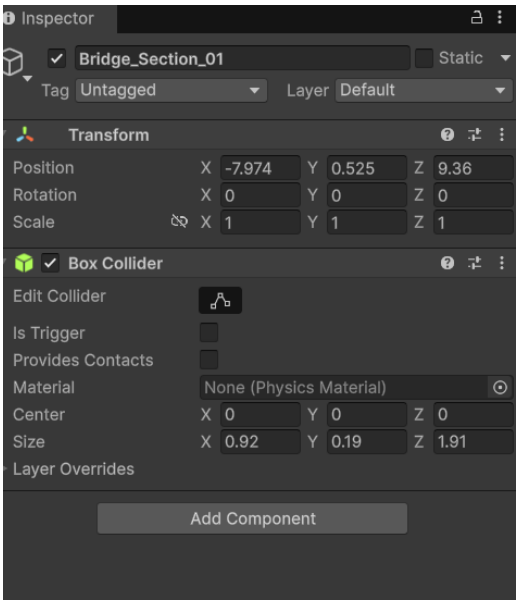


Figure 6 - Bridge Collision Behaviour

6. Cave victory trigger

The cave victory trigger is implemented through CaveWinTrigger.cs. This feature checks when the rabbit reaches the designated cave area and provides the player with completion feedback. The feature was tested in Unity, and the evidence includes the trigger script, a Console confirmation and the victory screen showing the level-completed message and Play Again button.



Figure 7 - CaveWinTrigger.cs

The WinTrigger was the most difficult part of my contribution. Testing this feature involved checking that the trigger was reached in the game and that the completion feedback appeared. This feature gives the player a clear objective and a visible result when the level is completed.

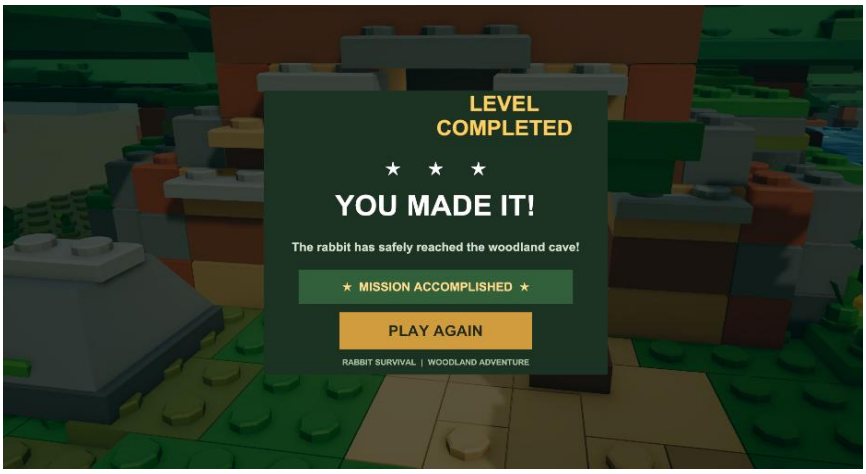


Figure 8 - CaveWinTrigger

7. Testing and debugging

I tested the four areas of my contribution in Unity: falling-rock behaviour, rabbit health and damage, bridge collision behaviour, and the cave victory trigger. The evidence set includes the relevant scripts, the falling-rock prefab physics configuration, Console output for health and defeat, bridge collider settings, and the victory interface.

The testing confirmed that the features could be exercised in Play Mode. The WinTrigger required the most attention during testing. For the final submission, the screenshots and the gameplay recording should be used to support the descriptions and show the visible outcomes. Any numerical values or detailed expected-versus-observed results should match the final project build.

8. GitHub contribution and collaboration

The project repository is available at https://github.com/iSaTHsiNDuMe/rabbit-carrot-collector-cw1. The individual branch supplied for this submission is https://github.com/iSaTHsiNDuMe/rabbit-carrot-collector-cw1/tree/student-1/falling-rocks.

The supplied GitHub screenshot shows the student-1/falling-rocks branch and a recent commit titled “Implemented Cave Victory UI”. The branch page also displays the project history. This provides supporting evidence of the work recorded in the individual branch. The group project was tested as a playable Unity prototype.

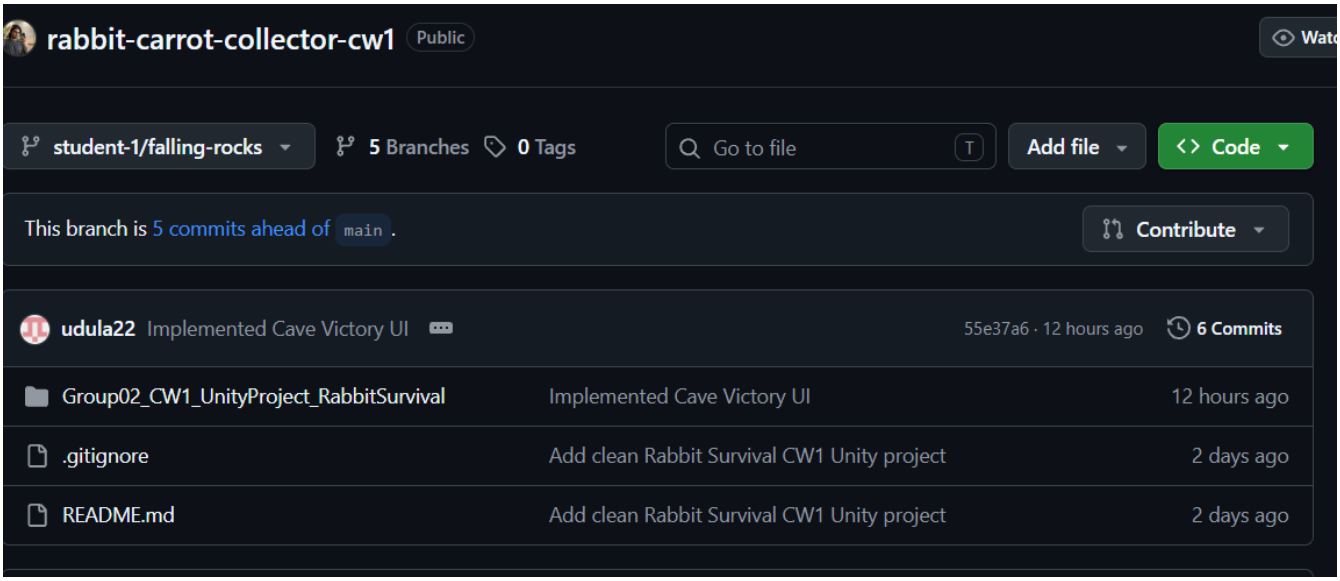


Figure 9 - GithubContribution

9. Personal reflection

My most difficult feature was the WinTrigger. Working through it helped me understand how trigger colliders can be used to detect when a player reaches a particular area and how that event can be connected to the game's completion feedback. I also learned more about using Box Colliders, key-binding controls and Rigidbody physics in Unity.

If I had more time, I would improve my animation skills and expand the scope of the game. I would also carry out more continuous playtesting across the complete route so that interactions can be checked together rather than only as separate features.

Evidence and supporting links

Gameplay recording:

https://drive.google.com/file/d/1jS9WU8Im7t8N3fnn_dOq50nIdpnvBe3v/view?usp=sharing

GitHub repository:

https://github.com/iSaTHsiNDuMe/rabbit-carrot-collector-cw1

Individual branch:

https://github.com/iSaTHsiNDuMe/rabbit-carrot-collector-cw1/tree/student-1/falling-rocks

10. AI assistance summary

During the development of Rabbit Survival LEGO, I used ChatGPT as a supporting tool to improve my understanding of Unity and C# scripting. It helped me troubleshoot technical issues, particularly a font compatibility problem affecting the user interface, and provided guidance when working with scripts related to the falling-rock system, rabbit health and damage, and the cave victory trigger. I also used AI assistance to better understand Unity components such as Box Colliders, Rigidbody physics and trigger interactions. This support helped me approach debugging more effectively and understand how different gameplay systems interact within the project. I tested the relevant features in Unity and used the results to check whether the solutions worked as intended. Overall, AI assistance was useful as a learning and troubleshooting resource throughout the development process, while the implementation and final testing remained part of my own project work.